

Module 3: The 70% Problem and Real Workflows

Structured workflows for partially correct output

Module 3: The 70% Problem and Real Workflows

Primary sources: Osmani, Chapter 3; Thomas & Hunt, Topic 12: Tracer Bullets.

Learning Outcomes

- Describe the 70 percent problem.
 - Apply structured workflows to guide generation.
 - Compare naive prompting with structured approaches.
 - Implement a basic feature using an AI-assisted workflow.
-

What Is the 70 Percent Problem?

AI output often looks close enough to create confidence but not close enough to trust.

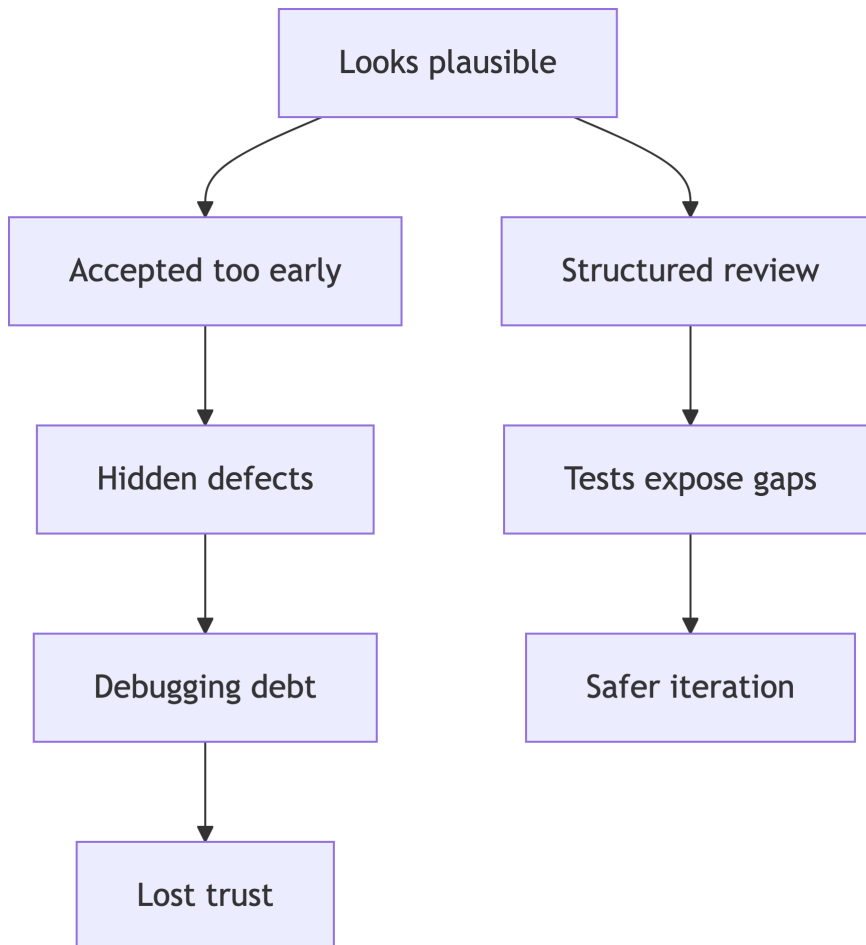
Osmani’s core warning uses the “70 percent” idea as a heuristic: AI can often produce the patterned, visible part of a solution, while edge cases, architecture, maintainability, and production readiness remain human engineering work.

Common shape:

- Compiles, but misses edge cases.
- Solves the sample, but not the requirement.
- Uses plausible APIs incorrectly.

- Produces code that is hard to maintain.
-

The 70 Percent Trap



Why It Is Dangerous

Partially correct output can be worse than obviously wrong output.

- It saves time early.

- It shifts cost into debugging and review.
- It creates sunk-cost pressure.
- It can hide security or reliability defects.

Common failure modes include the “two steps back” loop and the demo-quality trap: an impressive happy path that collapses under real use.

Real Workflow Patterns

Osmani describes three useful AI-assisted workflow patterns.

- AI as first drafter: generate, then refine, refactor, and test.
- AI as pair programmer: work in tight feedback loops with frequent review.
- AI as validator: write code first, then use AI to critique, test, and improve it.

The pattern matters less than the discipline around it: review, test, commit small, and understand what you merge.

Workflow Over Heroics

Effective AI-assisted development uses a loop:

1. Define intent.
 2. Ask for a small implementation.
 3. Inspect the diff.
 4. Run tests.
 5. Refine prompt or code.
 6. Document decisions.
-

Tracer Bullets

Thomas and Hunt recommend building thin, end-to-end slices that prove direction.

In AI-assisted SWE:

- Ask AI to help create the smallest complete path.
- Keep scope narrow enough to evaluate.
- Prefer inspectable progress over broad unfinished generation.
- Use the slice to get feedback under real project constraints.

Tracer code is not throwaway code. It is a lean but production-shaped skeleton that can grow as the system becomes clearer.

Golden Rules for the Remaining Work

Useful guardrails from Osmani Chapter 3:

- Be specific and clear about what you want.
 - Validate AI output against your intent.
 - Treat AI as a junior developer with supervision.
 - Use AI to expand capability, not replace thinking.
 - Isolate AI changes in version control.
 - Do not merge code you do not understand.
 - Document decisions, prompts, and useful patterns.
-

Bad AI Workflow

- Ask for a whole feature.
 - Paste output without reading.
 - Fix compile errors manually.
 - Skip tests because it “seems right.”
 - Document only the final result.
-

Better AI Workflow

- Ask for an implementation plan first.
 - Generate one component at a time.
 - Require tests before or alongside code.
 - Compare output against criteria.
 - Preserve rejected suggestions and rationale.
-

Lab 2 Development Connection

Eval-driven development starts this week.

Your lab goal is to make correctness measurable before relying on AI output.

Evaluation criteria should be:

- Specific.
 - Runnable or inspectable.
 - Connected to task intent.
 - Useful for regression.
-

Practice Activity

Given a Java feature request, define:

- One happy-path test.
- Two edge-case tests.
- One invalid-input test.
- One maintainability criterion.

Then ask Copilot to implement only enough code to pass those checks.

Takeaways

- The last 30 percent is engineering work.
- Small slices expose uncertainty early.
- AI is most useful when surrounded by feedback loops.